



Pensamento Computacional e Algoritmos

Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas

Professor: Roberto Wanderley da Nóbrega

Semestre: 2026.1

NOME: _____	NOTA
DATA:	

Prova 3

Atenção: A prova é individual e sem consulta. Não é permitido o acesso à Internet. Não é permitido o uso de ferramentas de inteligência artificial.

1. (4,0) (RaizDigital.java) A **raiz digital** de um número inteiro positivo é o dígito obtido somando-se repetidamente os seus dígitos: somam-se os dígitos do número, depois os dígitos desse resultado, e assim por diante, até restar um único dígito. O número de somas realizadas nesse processo é chamado de **persistência aditiva**. Escreva um programa que **LÊ DO TECLADO** um número inteiro positivo e **ESCREVE NA TELA** a cadeia de reduções (do número original até a raiz digital), a persistência aditiva e a raiz digital. Por exemplo:

- Para 9875, imprime a cadeia $9875 \rightarrow 29 \rightarrow 11 \rightarrow 2$, com persistência aditiva 3 e raiz digital 2.
- Para 12345, imprime a cadeia $12345 \rightarrow 15 \rightarrow 6$, com persistência aditiva 2 e raiz digital 6.
- Para 7, que já é um único dígito, imprime apenas 7, com persistência aditiva 0 e raiz digital 7.

Para resolver este problema, você deverá **IMPLEMENTAR O MÉTODO** `somaDigitos`, que recebe um número **inteiro** e retorna a soma de seus dígitos.

Restrições

- Não utilize arranjos (arrays).
- Apenas o método `main` deve realizar entrada e saída de dados e utilizar strings.

Dica

- Os dígitos de um número podem ser percorridos com divisão inteira e resto da divisão por 10.



Exemplos de execução

Entre com um inteiro positivo: 9875
9875 -> 29 -> 11 -> 2
Persistência aditiva: 3
Raiz digital: 2

Entre com um inteiro positivo: 7
7
Persistência aditiva: 0
Raiz digital: 7

2. (6,0) (MediaAparada.java) A **média aparada** de um conjunto de n números é obtida removendo-se, um a um, os m valores mais **discrepantes** — isto é, os mais distantes da média — e calculando-se a média aritmética simples dos valores restantes. Assuma $0 \leq m < n$. **Atenção:** a cada remoção, a média de referência deve ser **recalculada**, pois ela muda à medida que valores são descartados. Por exemplo, considere os $n = 5$ números

100, 10, 11, 12, 14

e $m = 2$. A média inicial é $(100 + 10 + 11 + 12 + 14)/5 = 29.4$; o valor mais distante dela é o 100, que é descartado. Restam 10, 11, 12, 14, cuja média é $(10 + 11 + 12 + 14)/4 = 11.75$; agora o mais distante é o 14, que é descartado. Restam então 10, 11, 12, e a média aparada é $(10 + 11 + 12)/3 = 11.0$. Observe que, na segunda remoção, foi descartado o 14 (e não o 10): por isso a média precisa ser recalculada a cada passo. *Dica:* A distância de um valor x à média μ é $|x - \mu|$; utilize `Math.abs`.

Para resolver este problema, você deverá **DEFINIR OS SEGUINTE MÉTODOS**:

- `calculaMedia`, que recebe um arranjo de reais e retorna a média aritmética de seus elementos.
- `encontraIndiceMaisDistante`, que recebe um arranjo de reais e retorna o índice (inteiro) do elemento mais distante da média. Em caso de empate, retorne o menor índice.
- `removeElemento`, que recebe um arranjo de reais e um índice inteiro e retorna uma **cópia** desse arranjo com o elemento no índice especificado removido, sem alterar o arranjo original. Por exemplo, se o arranjo for {100, 10, 11, 12, 14} e o índice for 0, então o método deve retornar o arranjo {10, 11, 12, 14}.
- `calculaMediaAparada`, que recebe um arranjo de reais e um inteiro m e retorna a média aparada dos elementos do arranjo, aparando m valores. Este método deve obter o resultado chamando os métodos `encontraIndiceMaisDistante` e `removeElemento` repetidamente.

Utilize o trecho de código abaixo como base. Não há necessidade de realizar entrada e saída de dados. Não utilize métodos prontos (por exemplo, da classe `java.util.Arrays`).

```
void main() {  
    double[] a = { 100, 10, 11, 12, 14 };  
    IO.println(calculaMediaAparada(a, 2)); // Deve imprimir 11.0  
    double[] b = { 1, 9, 5, 5 };  
    IO.println(calculaMediaAparada(b, 1)); // Deve imprimir 6.333333333333333  
    double[] c = { 9.5, 3.0, 4.5, 7.0 };  
    IO.println(calculaMediaAparada(c, 0)); // Deve imprimir 6.0  
}
```